

字符串算法专题

算法学习笔记

2026 年 5 月 9 日

目录

1	字符串基础	3
1.1	字符串匹配（暴力枚举）	3
1.2	字符串反转	4
1.3	字符串分割	5
2	字符串哈希	6
2.1	单哈希	6
2.2	双哈希	7
3	KMP 算法	9
3.1	字符串匹配	9
3.2	最小循环节	10
3.3	统计前缀出现次数	11
4	Trie 树（字典树）	13
4.1	字符串插入与查询	13
4.2	最大异或和（Trie 求异或极值）	15
5	Manacher 算法	17
5.1	最长回文子串	17
5.2	统计回文子串个数	18
6	后缀数组	19
6.1	后缀排序	19
6.2	最长公共前缀（height 数组）	21
6.3	不同子串个数	23
6.4	最长公共子串（两个字符串）	24

7 扩展 KMP (Z 算法)	26
7.1 计算 Z 数组	26
7.2 字符串匹配 (Z 算法)	27
8 AC 自动机	29
8.1 多模式串匹配	29
8.2 拓扑优化统计 (避免重复跳 fail)	31
9 回文自动机 (PAM)	33
9.1 构建回文自动机	33
10 后缀自动机 (SAM)	35
10.1 构建后缀自动机	35
10.2 最长公共子串 (多字符串, SAM)	37
11 最小表示法	40
11.1 循环同构最小表示	40
12 字符串算法总结	41

1 字符串基础

1.1 字符串匹配（暴力枚举）

解决问题：给定文本串 s 和模式串 p ，找出 p 在 s 中所有出现的位置。适用于小数据量、对时间要求不高的场景，或作为理解高级匹配算法的前置知识。

题目描述

给定一个文本串 s 和一个模式串 p ，请找出模式串 p 在文本串 s 中所有出现的位置（下标从 0 开始）。

输入示例

```
ababababa
aba
```

输出示例

```
0 2 4 6
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size(); // n: 文本串长度, m: 模式串长度
7          // 枚举所有可能的起始位置i, 检查从s[i]开始的长度为m的子串是否与p匹配
8          for (int i = 0; i + m <= n; i++) { // i: 当前匹配的起始位置
9              bool flag = true; // flag: 标记以i开头的子串是否匹配成功
10             for (int j = 0; j < m; j++) { // j: 模式串中当前比较的字符位置
11                 if (s[i + j] != p[j]) { // 当前字符不相等, 匹配失败
12                     flag = false;
13                     break;
14                 }
15             }
16             if (flag) cout << i << " "; // 匹配成功, 输出起始位置
17         }
18         cout << "\n";
19         return 0;
20     }
```

20

}

1.2 字符串反转

解决问题：反转一个字符串或判断一个字符串是否为回文串。适用于文本处理、回文判断等基础操作。

题目描述

给定一个字符串 s ，输出其反转后的字符串，并判断 s 是否为回文串。

输入示例

abccba

输出示例

abccba

Yes

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入的原始字符串
5          cin >> s;
6          string t = s; // t: 存储原始字符串的副本，用于后续比较
7          // 使用双指针法原地反转字符串
8          int n = s.size(); // n: 字符串长度
9          for (int i = 0; i < n / 2; i++) { // i: 左指针，从左向右移动
10             swap(s[i], s[n - 1 - i]); // n-1-i: 右指针，从右向左移动，交换两端
11                                     字符
12         }
13         cout << s << "\n"; // 输出反转后的字符串
14         if (t == s) cout << "Yes\n"; // 反转后与原串相等则为回文串
15         else cout << "No\n";
16         return 0;
17     }
```

1.3 字符串分割

解决问题：按照指定分隔符将字符串分割为多个子串列表。适用于处理 CSV 文件、解析命令参数、提取单词等场景。

题目描述

给定一个字符串 s 和一个分隔符 ch ，将 s 按 ch 分割成多个子串并输出。

输入示例

```
hello,world,string,split
,
```

输出示例

```
hello
world
string
split
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 待分割的原始字符串
5          char ch; // ch: 分隔符字符
6          cin >> s >> ch;
7          vector<string> res; // res: 存储分割后的所有子串
8          string cur = ""; // cur: 当前正在构建的子串
9          for (int i = 0; i < s.size(); i++) { // i: 遍历字符串的索引
10             if (s[i] == ch) { // 遇到分隔符，将当前积累的子串存入结果，并清空
11                 cur
12                 if (!cur.empty()) res.push_back(cur);
13                 cur = "";
14             } else {
15                 cur += s[i]; // 普通字符追加到当前子串末尾
16             }
17         }
18         if (!cur.empty()) res.push_back(cur); // 处理最后一个子串（末尾无分隔符）
```

```

18         for (auto &str : res) cout << str << "\n"; // 输出分割结果
19         return 0;
20     }

```

2 字符串哈希

2.1 单哈希

解决问题：快速进行字符串相等比较，支持 $O(1)$ 时间内判断任意两个子串是否相等。适用于字符串匹配、最长公共前缀、回文判定的预处理等。

题目描述

给定一个字符串 s 和 q 次询问，每次询问给定四个整数 $l1, r1, l2, r2$ ，判断子串 $s[l1..r1]$ 和 $s[l2..r2]$ 是否相等。

输入示例

```

abacaba
3
0 2 4 6
0 1 2 3
1 3 2 4

```

输出示例

```

Yes
No
No

```

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     typedef unsigned long long ull;
4     const int BASE = 131; // BASE: 哈希基数，通常取大于字符集且为质数的数
5     // 计算子串s[l..r]（闭区间）的哈希值
6     ull getHash(int l, int r, vector<ull>& h, vector<ull>& p) {
7         if (l == 0) return h[r]; // 左端点为0时直接返回前缀哈希
8         return h[r] - h[l - 1] * p[r - l + 1]; // 通过前缀哈希相减得到子串哈希

```

```
9     }
10     int main() {
11         string s; // s: 待处理的字符串
12         int q;    // q: 询问次数
13         cin >> s >> q;
14         int n = s.size(); // n: 字符串长度
15         vector<ull> h(n), p(n); // h[i]: 前缀[0..i]的哈希值, p[i]: BASE的i次
                                // 幂
16         h[0] = s[0]; // 初始化第一个字符的哈希值
17         p[0] = 1;    // BASE^0 = 1
18         for (int i = 1; i < n; i++) {
19             h[i] = h[i - 1] * BASE + s[i]; // 递推计算前缀哈希
20             p[i] = p[i - 1] * BASE;        // 递推计算BASE的幂次
21         }
22         while (q--) {
23             int l1, r1, l2, r2; // 询问的两个子串区间[l1,r1]和[l2,r2]
24             cin >> l1 >> r1 >> l2 >> r2;
25             if (getHash(l1, r1, h, p) == getHash(l2, r2, h, p))
26                 cout << "Yes\n";
27             else
28                 cout << "No\n";
29         }
30         return 0;
31     }
```

2.2 双哈希

解决问题：在单哈希基础上使用两个不同的基数和模数计算哈希值，有效降低哈希冲突概率。适用于对正确性要求极高、无法承受哈希碰撞的场景。

题目描述

同单哈希题目描述，询问两个子串是否相等，使用双哈希提高可靠性。

输入示例

```
abacaba
3
0 2 4 6
0 1 2 3
```

1 3 2 4

输出示例

Yes

No

No

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      typedef long long ll;
4      const ll BASE1 = 131, MOD1 = 1e9 + 7; // 第一组哈希的参数: 基数B1, 模数M1
5      const ll BASE2 = 13331, MOD2 = 998244353; // 第二组哈希的参数: 基数B2, 模数M2
6      struct HashPair {
7          ll h1, h2; // h1: 第一组哈希值, h2: 第二组哈希值
8          bool operator==(const HashPair& o) const {
9              return h1 == o.h1 && h2 == o.h2; // 两组哈希值都相等才判定为真
10         }
11     };
12     int main() {
13         string s; // s: 待处理的字符串
14         int q; // q: 询问次数
15         cin >> s >> q;
16         int n = s.size(); // n: 字符串长度
17         vector<HashPair> h(n); // h[i]: 前缀[0..i]的双哈希值
18         vector<ll> p1(n), p2(n); // p1[i]: B1^i mod M1, p2[i]: B2^i mod M2
19         h[0] = {s[0] % MOD1, s[0] % MOD2};
20         p1[0] = p2[0] = 1;
21         for (int i = 1; i < n; i++) {
22             auto& cur = h[i], &pre = h[i-1];
23             cur.h1 = (pre.h1 * BASE1 + s[i]) % MOD1; // 递推计算第一组哈希
24             cur.h2 = (pre.h2 * BASE2 + s[i]) % MOD2; // 递推计算第二组哈希
25             p1[i] = (p1[i-1] * BASE1) % MOD1; // 递推计算基数的幂次
26             p2[i] = (p2[i-1] * BASE2) % MOD2;
27         }
28         // 查询闭区间[l,r]子串的双哈希值
29         auto getHash = [&](int l, int r) -> HashPair {

```



```

30     if (l == 0) return h[r];
31     HashPair res;
32     res.h1 = (h[r].h1 - h[l-1].h1 * p1[r-l+1] % MOD1 + MOD1) % MOD1;
           // 减去左端贡献, 加模防负
33     res.h2 = (h[r].h2 - h[l-1].h2 * p2[r-l+1] % MOD2 + MOD2) % MOD2;
34     return res;
35 };
36 while (q--) {
37     int l1, r1, l2, r2; // 询问的两个子串区间
38     cin >> l1 >> r1 >> l2 >> r2;
39     if (getHash(l1, r1) == getHash(l2, r2))
40         cout << "Yes\n";
41     else
42         cout << "No\n";
43 }
44 return 0;
45 }

```

3 KMP 算法

3.1 字符串匹配

解决问题：在线性时间复杂度 $O(n+m)$ 内找出模式串在文本串中的所有出现位置。适用于需要高效匹配的任何场景，如文本编辑器中的查找功能、日志分析等。

题目描述

给定文本串 s 和模式串 p ，输出 p 在 s 中每次出现的起始下标（下标从 0 开始）。

输入示例

```

ababcabababab
abab

```

输出示例

```

0 7

```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size(); // n: 文本串长度, m: 模式串长度
7          vector<int> nxt(m); // nxt[i]: 模式串前缀p[0..i]的最长相等前后缀长度
            (部分匹配表)
8          // 构建next数组: nxt[i]表示p[0..i]中, 既是前缀又是后缀的最长子串长度
9          nxt[0] = 0; // 长度为1的前缀, 没有非平凡的前后缀, 填0
10         for (int i = 1, j = 0; i < m; i++) { // i: 当前正在计算nxt的后缀末尾下
            标, j: 当前匹配的前缀末尾下标
11             while (j > 0 && p[i] != p[j]) j = nxt[j - 1]; // 失配时回退j, 利用
            已求的nxt信息
12             if (p[i] == p[j]) j++; // 当前字符匹配, 前缀长度+1
13             nxt[i] = j; // 记录nxt值
14         }
15         // 利用next数组进行匹配
16         for (int i = 0, j = 0; i < n; i++) { // i: 文本串当前扫描位置, j: 模
            式串当前匹配位置
17             while (j > 0 && s[i] != p[j]) j = nxt[j - 1]; // 失配时, 根据next
            数组回退模式串指针
18             if (s[i] == p[j]) j++; // 当前位置匹配成功, j后移
19             if (j == m) { // 完全匹配模式串
20                 cout << i - m + 1 << " "; // 输出匹配起始位置
21                 j = nxt[j - 1]; // 继续寻找下一次匹配
22             }
23         }
24         cout << "\n";
25         return 0;
26     }
```

3.2 最小循环节

解决问题: 利用 KMP 的 next 数组找出字符串的最小循环节。适用于字符串周期分析、拼接字符串最小次数等场景。

题目描述

给定一个字符串 s ，判断其是否可以由一个较短的子串重复多次构成。若是，输出该最短重复子串；否则输出原串本身。

输入示例

abccabccabcc

输出示例

abc

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          int n = s.size(); // n: 字符串长度
7          vector<int> nxt(n);
8          nxt[0] = 0;
9          for (int i = 1, j = 0; i < n; i++) {
10             while (j > 0 && s[i] != s[j]) j = nxt[j - 1];
11             if (s[i] == s[j]) j++;
12             nxt[i] = j;
13         }
14         int len = n - nxt[n - 1]; // len: 最小循环节的长度
15         // 如果len能整除n, 则字符串可以由循环节重复构造
16         if (n % len == 0) cout << s.substr(0, len) << "\n";
17         else cout << s << "\n"; // 不能整除, 说明没有完整循环节, 输出原串
18         return 0;
19     }
```

3.3 统计前缀出现次数

解决问题：统计模式串的每个前缀在文本串中出现的次数。适用于统计单词前缀频次等场景。

题目描述

给定文本串 s 和模式串 p ，输出 p 的每个前缀在 s 中出现的次数。

输入示例

ababa

aba

输出示例

前缀a: 3

前缀ab: 2

前缀aba: 2

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size();
7          vector<int> nxt(m);
8          nxt[0] = 0;
9          for (int i = 1, j = 0; i < m; i++) {
10             while (j > 0 && p[i] != p[j]) j = nxt[j - 1];
11             if (p[i] == p[j]) j++;
12             nxt[i] = j;
13         }
14         vector<int> cnt(m + 1, 0); // cnt[i]: 模式串长度为i的前缀在文本串中出现的次数
15         for (int i = 0, j = 0; i < n; i++) { // 在文本串中匹配模式串
16             while (j > 0 && s[i] != p[j]) j = nxt[j - 1];
17             if (s[i] == p[j]) j++;
18             cnt[j]++; // 当前匹配到的前缀长度j出现一次
19         }
20         // 从长到短累加计数: 若长前缀出现过, 其短前缀也一定出现
21         for (int i = m; i >= 1; i--) {
22             if (nxt[i - 1] > 0) cnt[nxt[i - 1]] += cnt[i];
23         }
```

```
24     for (int i = 1; i <= m; i++) {  
25         cout << "前缀" << p.substr(0, i) << ": " << cnt[i] << "\n";  
26     }  
27     return 0;  
28 }
```

4 Trie 树（字典树）

4.1 字符串插入与查询

解决问题：高效存储和检索大量字符串集合，支持快速查询某个字符串是否出现及其出现次数。适用于搜索引擎关键词补全、IP 路由查找、词频统计等场景。

题目描述

给定 n 个单词进行建树，随后有 q 次询问，每次询问一个单词是否在集合中出现过以及出现次数。

输入示例

```
5  
apple  
app  
apple  
banana  
orange  
3  
apple  
app  
grape
```

输出示例

```
apple found, count: 2  
app found, count: 1  
grape not found
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 100005; // MAXN: 最大节点数量 (根据字符串总长度估算)
4      int trie[MAXN][26]; // trie[u][c]: 节点u通过字符c (映射为0~25) 指向的子节点编号
5
6      int cnt[MAXN]; // cnt[u]: 以节点u结尾的字符串出现次数
7      int tot = 1; // tot: 当前已分配的节点总数, 根节点为1
8      // 向字典树中插入字符串s
9      void insert(const string& s) {
10         int u = 1; // u: 当前所在节点编号, 从根节点开始
11         for (int i = 0; i < s.size(); i++) { // i: 遍历字符串中字符的位置
12             int c = s[i] - 'a'; // c: 当前字符映射为0~25的整数
13             if (!trie[u][c]) trie[u][c] = ++tot; // 若边不存在, 创建新节点
14             u = trie[u][c]; // 沿边移动到下一节点
15         }
16         cnt[u]++; // 在字符串结尾节点增加计数
17     }
18     // 查询字符串s的出现次数, 返回次数 (0表示未出现)
19     int query(const string& s) {
20         int u = 1;
21         for (int i = 0; i < s.size(); i++) {
22             int c = s[i] - 'a';
23             if (!trie[u][c]) return 0; // 某字符边不存在, 直接返回未找到
24             u = trie[u][c];
25         }
26         return cnt[u]; // 返回结尾节点的计数值
27     }
28     int main() {
29         int n, q; // n: 插入单词数量, q: 查询单词数量
30         cin >> n;
31         string word; // word: 临时存储输入的单词
32         for (int i = 0; i < n; i++) {
33             cin >> word;
34             insert(word);
35         }
36         cin >> q;
37         for (int i = 0; i < q; i++) {
38             cin >> word;
39             int res = query(word); // res: 查询结果, 即该单词出现次数
```

```
39         if (res > 0) cout << word << " found, count: " << res << "\n";
40         else cout << word << " not found\n";
41     }
42     return 0;
43 }
```

4.2 最大异或和（Trie 求异或极值）

解决问题：在一组整数中快速找出与给定值异或结果最大的数。适用于网络编码、密码学、数据压缩等需要异或优化的场景。

题目描述

给定 n 个非负整数，有 q 次询问，每次询问一个整数 x ，求 x 与数组中任意元素异或的最大值。

输入示例

```
5
3 7 2 9 5
3
4 6 8
```

输出示例

```
13
14
15
```

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     const int MAXN = 100005 * 32; // 每个数最多32位，节点数约为数的个数*32
4     int trie[MAXN][2], tot = 1; // trie[u][0/1]: 节点u的0/1子节点编号，tot为
    当前节点总数
5     // 向二进制Trie中插入数x
6     void insert(int x) {
7         int u = 1; // u: 当前节点编号
8         for (int i = 30; i >= 0; i--) { // i: 当前处理的二进制位，从高位向低位
9             int b = (x >> i) & 1; // b: x在第i位的二进制值
```

```
10         if (!trie[u][b]) trie[u][b] = ++tot; // 创建新节点
11         u = trie[u][b]; // 移动到子节点
12     }
13 }
14 // 查询与x异或的最大值
15 int queryMaxXor(int x) {
16     int u = 1, res = 0; // res: 能获得的最大异或值
17     for (int i = 30; i >= 0; i--) {
18         int b = (x >> i) & 1;
19         if (trie[u][b ^ 1]) { // 优先选相反位, 使异或结果该位为1
20             res |= (1 << i); // 累加该位的贡献
21             u = trie[u][b ^ 1];
22         } else {
23             u = trie[u][b]; // 无奈只能选相同位
24         }
25     }
26     return res;
27 }
28 int main() {
29     int n, q; // n: 数组大小, q: 询问次数
30     cin >> n;
31     for (int i = 0; i < n; i++) {
32         int x; // x: 输入的元素
33         cin >> x;
34         insert(x);
35     }
36     cin >> q;
37     for (int i = 0; i < q; i++) {
38         int x; // x: 询问的值
39         cin >> x;
40         cout << queryMaxXor(x) << "\n";
41     }
42     return 0;
43 }
```


5 Manacher 算法

5.1 最长回文子串

解决问题：在线性时间 $O(n)$ 内求出字符串中最长的回文子串。适用于文本处理、DNA 序列分析、寻找对称结构等场景。

题目描述

给定一个字符串 s ，找出其最长的回文子串。如果有多个，返回任意一个。

输入示例

babad

输出示例

bab

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 原始输入字符串
5          cin >> s;
6          // 预处理：在原字符串每个字符间插入特殊字符'#'，便于统一奇偶长度回文
7          string t = "#"; // t: 处理后的字符串，以'#'开头
8          for (int i = 0; i < s.size(); i++) {
9              t += s[i];
10             t += '#'; // 字符间和末尾加'#'
11         }
12         int n = t.size(); // n: 处理后字符串长度
13         vector<int> p(n); // p[i]: 以t[i]为中心的最长回文半径（包含中心自身）
14         int c = 0, r = 0; // c: 当前最右回文串的中心, r: 该回文串的右边界
15         for (int i = 0; i < n; i++) {
16             // 利用对称性初始化p[i]: i关于c的对称点为2*c-i, 取min避免越界
17             if (i < r) p[i] = min(r - i, p[2 * c - i]);
18             else p[i] = 1; // 至少包含自身
19             // 以i为中心向两侧扩展
20             while (i - p[i] >= 0 && i + p[i] < n && t[i - p[i]] == t[i + p[i]]) {
```

```
21         p[i]++; // 匹配成功, 半径加1
22     }
23     // 更新最右回文串的中心和右边界
24     if (i + p[i] > r) {
25         c = i;
26         r = i + p[i];
27     }
28 }
29 int center = 0, maxR = 0; // center: 最长回文子串的中心索引, maxR: 最
    长半径
30 for (int i = 0; i < n; i++) {
31     if (p[i] > maxR) {
32         maxR = p[i];
33         center = i;
34     }
35 }
36 // 从处理后字符串还原原始字符串中的回文子串
37 int start = (center - maxR + 1) / 2; // start: 原始字符串中的起始索引
38 int len = maxR - 1; // len: 原始回文子串长度
39 cout << s.substr(start, len) << "\n";
40 return 0;
41 }
```

5.2 统计回文子串个数

解决问题：在线性时间内统计字符串中所有回文子串的总数。适用于文本分析、模式识别等场景。

题目描述

给定一个字符串 s ，统计其中所有回文子串的数量（不同位置的相同字符串计为不同回文子串）。

输入示例

abc

输出示例

3

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          string t = "#";
7          for (int i = 0; i < s.size(); i++) {
8              t += s[i];
9              t += '#';
10         }
11         int n = t.size();
12         vector<int> p(n);
13         int c = 0, r = 0;
14         long long ans = 0; // ans: 回文子串总数
15         for (int i = 0; i < n; i++) {
16             if (i < r) p[i] = min(r - i, p[2 * c - i]);
17             else p[i] = 1;
18             while (i - p[i] >= 0 && i + p[i] < n && t[i - p[i]] == t[i + p[i]]) {
19                 p[i]++;
20             }
21             if (i + p[i] > r) {
22                 c = i;
23                 r = i + p[i];
24             }
25             ans += p[i] / 2; // 以该位置为中心的回文子串数等于半径的一半（向下取整）
26         }
27         cout << ans << "\n";
28         return 0;
29     }
```

6 后缀数组

6.1 后缀排序

解决问题：对字符串的所有后缀按字典序进行排序，得到后缀排名数组和后缀数组。是许多高级字符串处理问题（最长公共前缀、不同子串个数等）的基础。

题目描述

给定一个字符串 s ，输出其所有后缀按字典序排序后的起始索引（后缀数组 sa ）以及每个后缀的排名（排名数组 rk ）。

输入示例

ababa

输出示例

sa: 4 2 0 3 1

rk: 2 4 1 3 0

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          int n = s.size(); // n: 字符串长度
7          vector<int> sa(n), rk(n << 1), oldrk(n << 1); // sa[i]: 排名为i的后缀
           起始索引; rk[i]: 后缀i的排名
8          // 初始按单字符排序（基数排序思想）
9          for (int i = 0; i < n; i++) {
10             sa[i] = i;
11             rk[i] = s[i]; // 初始排名即字符的ASCII码值
12         }
13         // 倍增法：每次将排序依据从长度为k的后缀扩展到2k
14         for (int k = 1; k < n; k <= 1) { // k: 当前已排序前缀的长度
15             // 自定义比较器：先比较第一关键字rk[i]，再比较第二关键字rk[i+k]
16             auto cmp = [&](int i, int j) -> bool {
17                 if (rk[i] != rk[j]) return rk[i] < rk[j];
18                 int ri = (i + k < n) ? rk[i + k] : -1; // 若第二关键字越界视为
                    最小
19                 int rj = (j + k < n) ? rk[j + k] : -1;
20                 return ri < rj;
21             };
22             sort(sa.begin(), sa.end(), cmp);
23             // 重新计算排名
```

```

24     oldrk = rk; // oldrk: 保存上一轮的排名, 用于比较
25     for (int i = 0, cnt = 0; i < n; i++) {
26         if (i > 0 && oldrk[sa[i]] == oldrk[sa[i-1]] &&
27             (sa[i]+k < n ? oldrk[sa[i]+k] : -1) == (sa[i-1]+k < n ? oldrk[
28                 sa[i-1]+k] : -1))
29             rk[sa[i]] = cnt; // 两后缀当前长度相同, 排名不变
30         else
31             rk[sa[i]] = ++cnt; // 长度不同, 排名递增
32     }
33     cout << "sa: ";
34     for (int i = 0; i < n; i++) cout << sa[i] << " \n"[i == n - 1];
35     cout << "rk: ";
36     for (int i = 0; i < n; i++) cout << rk[i] << " \n"[i == n - 1];
37     return 0;
38 }

```

6.2 最长公共前缀 (height 数组)

解决问题: 在得到后缀数组后, 在线性时间内计算相邻排名后缀的最长公共前缀 (LCP)。可用于求两个后缀的 LCP、不同子串个数等。

题目描述

给定一个字符串 s , 输出后缀数组 sa 和相邻后缀的最长公共前缀数组 $height$ 。

输入示例

ababa

输出示例

sa: 4 2 0 3 1

height: 0 1 3 0 2

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s; // s: 输入字符串

```

```

5      cin >> s;
6      int n = s.size();
7      vector<int> sa(n), rk(n << 1), oldrk(n << 1);
8      for (int i = 0; i < n; i++) {
9          sa[i] = i;
10         rk[i] = s[i];
11     }
12     for (int k = 1; k < n; k <= 1) {
13         auto cmp = [&](int i, int j) -> bool {
14             if (rk[i] != rk[j]) return rk[i] < rk[j];
15             int ri = (i + k < n) ? rk[i + k] : -1;
16             int rj = (j + k < n) ? rk[j + k] : -1;
17             return ri < rj;
18         };
19         sort(sa.begin(), sa.end(), cmp);
20         oldrk = rk;
21         for (int i = 0, cnt = 0; i < n; i++) {
22             if (i > 0 && oldrk[sa[i]] == oldrk[sa[i-1]] &&
23                 (sa[i]+k < n ? oldrk[sa[i]+k] : -1) == (sa[i-1]+k < n ? oldrk[
24                     sa[i-1]+k] : -1))
25                 rk[sa[i]] = cnt;
26             else
27                 rk[sa[i]] = ++cnt;
28         }
29         vector<int> height(n); // height[i]: 排名为i的后缀与排名为i-1的后缀的
30                                // 最长公共前缀长度
31         for (int i = 0, k = 0; i < n; i++) { // k: 当前计算出的LCP长度
32             if (rk[i] == 0) { height[0] = 0; continue; } // 排名第一的后缀没有
33             // 前一名
34             if (k) k--; // 利用性质: 相邻后缀的LCP至少为上一对LCP-1
35             int j = sa[rk[i] - 1]; // j: 排名比i前一位的后缀起始位置
36             while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++; // 暴
37             // 力扩展LCP
38             height[rk[i]] = k; // 记录排名rk[i]处的height值
39         }
40         cout << "sa: ";
41         for (int i = 0; i < n; i++) cout << sa[i] << " \n"[i == n - 1];
42         cout << "height: ";
43         for (int i = 0; i < n; i++) cout << height[i] << " \n"[i == n - 1];

```

```
41     return 0;
42 }
```

6.3 不同子串个数

解决问题：利用后缀数组和 height 数组统计字符串中本质不同的子串总数。适用于文本分析、重复模式检测等场景。

题目描述

给定一个字符串 s ，求其本质不同的子串个数。

输入示例

abc

输出示例

6

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      string s; // s: 输入字符串
5      cin >> s;
6      int n = s.size();
7      vector<int> sa(n), rk(n << 1), oldrk(n << 1);
8      for (int i = 0; i < n; i++) {
9          sa[i] = i;
10         rk[i] = s[i];
11     }
12     for (int k = 1; k < n; k <= 1) {
13         auto cmp = [&](int i, int j) -> bool {
14             if (rk[i] != rk[j]) return rk[i] < rk[j];
15             int ri = (i + k < n) ? rk[i + k] : -1;
16             int rj = (j + k < n) ? rk[j + k] : -1;
17             return ri < rj;
18         };
19         sort(sa.begin(), sa.end(), cmp);
```

```

20     oldrk = rk;
21     for (int i = 0, cnt = 0; i < n; i++) {
22         if (i > 0 && oldrk[sa[i]] == oldrk[sa[i-1]] &&
23             (sa[i]+k < n ? oldrk[sa[i]+k] : -1) == (sa[i-1]+k < n ? oldrk[
24                 sa[i-1]+k] : -1))
25             rk[sa[i]] = cnt;
26         else
27             rk[sa[i]] = ++cnt;
28     }
29     vector<int> height(n);
30     for (int i = 0, k = 0; i < n; i++) {
31         if (rk[i] == 0) { height[0] = 0; continue; }
32         if (k) k--;
33         int j = sa[rk[i] - 1];
34         while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
35         height[rk[i]] = k;
36     }
37     long long ans = 0; // ans: 本质不同子串数量
38     for (int i = 0; i < n; i++) {
39         ans += n - sa[i] - height[i]; // 每个后缀贡献的子串数 = 后缀长度 -
40             与前一名重复的长度
41     }
42     cout << ans << "\n";
43     return 0;

```

6.4 最长公共子串（两个字符串）

解决问题：找出两个字符串的最长公共子串。适用于基因序列比对、文本查重、版本差异比较等场景。

题目描述

给定两个字符串 s_1 和 s_2 ，求它们的最长公共子串的长度和内容。

输入示例

```

babcd
abdec

```


输出示例

3 bab

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s1, s2; // s1: 第一个字符串, s2: 第二个字符串
5          cin >> s1 >> s2;
6          // 将两个字符串拼接, 中间用未出现的字符'#'分隔
7          string s = s1 + "#" + s2; // s: 拼接后字符串
8          int n = s.size(), n1 = s1.size(); // n: 拼接串长度, n1: 第一个字符串长
          度
9          vector<int> sa(n), rk(n << 1), oldrk(n << 1);
10         for (int i = 0; i < n; i++) {
11             sa[i] = i;
12             rk[i] = s[i];
13         }
14         for (int k = 1; k < n; k <= 1) {
15             auto cmp = [&](int i, int j) -> bool {
16                 if (rk[i] != rk[j]) return rk[i] < rk[j];
17                 int ri = (i + k < n) ? rk[i + k] : -1;
18                 int rj = (j + k < n) ? rk[j + k] : -1;
19                 return ri < rj;
20             };
21             sort(sa.begin(), sa.end(), cmp);
22             oldrk = rk;
23             for (int i = 0, cnt = 0; i < n; i++) {
24                 if (i > 0 && oldrk[sa[i]] == oldrk[sa[i-1]] &&
25                     (sa[i]+k < n ? oldrk[sa[i]+k] : -1) == (sa[i-1]+k < n ? oldrk[
26                         sa[i-1]+k] : -1))
27                     rk[sa[i]] = cnt;
28                 else
29                     rk[sa[i]] = ++cnt;
30             }
31         }
32         vector<int> height(n);
33         for (int i = 0, k = 0; i < n; i++) {
34             if (rk[i] == 0) { height[0] = 0; continue; }

```

```

34         if (k) k--;
35         int j = sa[rk[i] - 1];
36         while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
37         height[rk[i]] = k;
38     }
39     int maxlen = 0, start = 0; // maxlen: 最长公共子串长度, start: 该子串
    // 在s中的起始位置
40     for (int i = 1; i < n; i++) {
41         // 检查排名相邻的两个后缀是否分别来自不同原串 (一个在分隔符前, 一个
    // 在后)
42         if ((sa[i] < n1 && sa[i-1] > n1) || (sa[i] > n1 && sa[i-1] < n1))
43         {
44             if (height[i] > maxlen) {
45                 maxlen = height[i];
46                 start = sa[i];
47             }
48         }
49     }
50     cout << maxlen << " " << s.substr(start, maxlen) << "\n";
51     return 0;
}

```

7 扩展 KMP (Z 算法)

7.1 计算 Z 数组

解决问题：在线性时间内计算字符串每个后缀与原串前缀的最长公共前缀（Z 数组）。适用于字符串匹配、周期分析、Border 计算等场景。

题目描述

给定一个字符串 s ，输出其 Z 数组，其中 $z[i]$ 表示后缀 $s[i..]$ 与 s 的最长公共前缀长度。

输入示例

aabxaabx

输出示例

```
8 1 0 0 4 1 0 0
```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          int n = s.size(); // n: 字符串长度
7          vector<int> z(n); // z[i]: 以i起始的后缀与原串前缀的最长匹配长度
8          z[0] = n; // 整个字符串与自身完全匹配
9          // 维护一个区间[l, r], 表示当前已知的最靠右的匹配段
10         for (int i = 1, l = 0, r = 0; i < n; i++) { // l: 最右匹配段左端点, r
11             : 最右匹配段右端点
12             if (i <= r) z[i] = min(r - i + 1, z[i - l]); // 利用对称性初始化
13             while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++; // 暴力扩
14             展
15             if (i + z[i] - 1 > r) { // 更新最右匹配段
16                 l = i;
17                 r = i + z[i] - 1;
18             }
19         }
20         for (int i = 0; i < n; i++) {
21             cout << z[i] << " \n"[i == n - 1];
22         }
23         return 0;
24     }

```

7.2 字符串匹配 (Z 算法)

解决问题: 利用 Z 算法在线性时间内找出模式串在文本串中的所有出现位置。与 KMP 同为线性匹配算法, 实现方式不同。

题目描述

给定文本串 s 和模式串 p , 输出 p 在 s 中每次出现的起始下标。

输入示例

```
ababa
```

aba

输出示例

0 2

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          // 将模式串拼接在文本串前面, 用特殊字符分隔
7          string combined = p + "#" + s; // combined: 拼接后的字符串
8          int m = p.size(), n = combined.size(); // m: 模式串长度, n: 拼接串长
9          vector<int> z(n);
10         z[0] = n;
11         for (int i = 1, l = 0, r = 0; i < n; i++) {
12             if (i <= r) z[i] = min(r - i + 1, z[i - l]);
13             while (i + z[i] < n && combined[z[i]] == combined[i + z[i]]) z[i]
14                 ++;
15             if (i + z[i] - 1 > r) {
16                 l = i;
17                 r = i + z[i] - 1;
18             }
19         }
20         // 扫描拼接串中模式串之后的部分, z[i]等于m说明匹配成功
21         for (int i = m + 1; i < n; i++) { // i: 文本串部分对应的索引
22             if (z[i] == m) cout << i - m - 1 << " "; // 输出在文本串中的起始位
23             置
24         }
25         cout << "\n";
26         return 0;
27     }
```

8 AC 自动机

8.1 多模式串匹配

解决问题：给定一个文本串和一组模式串，一次性找出所有模式串在文本串中的出现位置。适用于敏感词过滤、搜索引擎关键词高亮、防病毒特征匹配等场景。

题目描述

给定一个文本串 s 和 k 个模式串，输出每个模式串在 s 中出现的次数。

输入示例

```
ahishers
3
he she his
```

输出示例

```
he: 1
she: 1
his: 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 500005; // MAXN: Trie最大节点数
4      int trie[MAXN][26]; // trie[u][c]: 节点u经字符c指向的子节点
5      int fail[MAXN]; // fail[u]: 节点u的失配指针
6      int cnt[MAXN]; // cnt[u]: 以节点u结尾的模式串的编号（或计数）
7      int tot = 1; // tot: 当前节点总数
8      // 向Trie中插入模式串，返回该模式串对应的终止节点编号
9      int insert(const string& s, int id) {
10         int u = 1; // u: 从根节点开始
11         for (int i = 0; i < s.size(); i++) {
12             int c = s[i] - 'a';
13             if (!trie[u][c]) trie[u][c] = ++tot;
14             u = trie[u][c];
15         }
16         if (!cnt[u]) cnt[u] = id; // 记录该终止节点对应的模式串编号
17         return u;
```

```

18     }
19     // 构建AC自动机的失配指针
20     void build() {
21         queue<int> q; // q: BFS队列
22         for (int c = 0; c < 26; c++) { // 初始化根节点的子节点fail指向根
23             int v = trie[1][c];
24             if (v) {
25                 fail[v] = 1;
26                 q.push(v);
27             }
28         }
29         while (!q.empty()) {
30             int u = q.front(); q.pop();
31             for (int c = 0; c < 26; c++) {
32                 int v = trie[u][c];
33                 if (v) {
34                     fail[v] = trie[fail[u]][c]; // 失配指针沿父节点fail的对应字
35                                                 符走
36                     q.push(v);
37                 } else {
38                     trie[u][c] = trie[fail[u]][c]; // 空子节点直接指向父节点
39                                                 fail的对应子节点
40                 }
41             }
42         }
43     }
44     int main() {
45         string s; // s: 文本串
46         int k; // k: 模式串数量
47         cin >> s >> k;
48         vector<string> patterns(k); // patterns[i]: 第i个模式串
49         vector<int> endNode(k); // endNode[i]: 第i个模式串在Trie中的终止节点
50         for (int i = 0; i < k; i++) {
51             cin >> patterns[i];
52             endNode[i] = insert(patterns[i], i + 1); // id从1开始
53         }
54         build();
55         vector<int> ans(k + 1, 0); // ans[id]: 编号为id的模式串在文本串中出现的
56                                     次数
57         int u = 1; // u: 当前匹配状态对应的Trie节点

```

```

55     for (int i = 0; i < s.size(); i++) {
56         int c = s[i] - 'a';
57         u = trie[u][c]; // 转移到下一状态（失配已预处理）
58         // 沿fail链统计所有匹配到的模式串
59         for (int temp = u; temp > 1; temp = fail[temp]) {
60             if (cnt[temp]) ans[cnt[temp]]++;
61         }
62     }
63     for (int i = 0; i < k; i++) {
64         cout << patterns[i] << ": " << ans[i + 1] << "\n";
65     }
66     return 0;
67 }

```

8.2 拓扑优化统计（避免重复跳 fail）

解决问题：在 AC 自动机中高效统计每个模式串的出现次数，避免每次匹配都沿 fail 链暴力跳转。当模式串很多且文本较长时，可将复杂度控制在 $O(n + \text{节点数})$ 。

题目描述

同多模式串匹配题目，但数据规模更大，需要优化统计过程。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const int MAXN = 500005;
4     int trie[MAXN][26], fail[MAXN];
5     int flag[MAXN]; // flag[u]: 节点u对应的模式串编号
6     int indeg[MAXN]; // indeg[u]: 节点u在fail树中的入度，用于拓扑排序
7     int sum[MAXN]; // sum[u]: 匹配过程中经过节点u的次数
8     int tot = 1;
9     int insert(const string& s, int id) {
10         int u = 1;
11         for (int i = 0; i < s.size(); i++) {
12             int c = s[i] - 'a';
13             if (!trie[u][c]) trie[u][c] = ++tot;
14             u = trie[u][c];
15         }
16         if (!flag[u]) flag[u] = id;
17         return u;
18     }

```

```
19 void build() {
20     queue<int> q;
21     for (int c = 0; c < 26; c++) {
22         int v = trie[1][c];
23         if (v) {
24             fail[v] = 1;
25             indeg[1]++; // 根节点入度增加
26             q.push(v);
27         }
28     }
29     while (!q.empty()) {
30         int u = q.front(); q.pop();
31         for (int c = 0; c < 26; c++) {
32             int v = trie[u][c];
33             if (v) {
34                 fail[v] = trie[fail[u]][c];
35                 indeg[fail[v]]++; // fail[v]的入度增加, 建立fail树的边
36                 q.push(v);
37             } else {
38                 trie[u][c] = trie[fail[u]][c];
39             }
40         }
41     }
42 }
43 int main() {
44     string s; // s: 文本串
45     int k;    // k: 模式串数量
46     cin >> s >> k;
47     vector<string> patterns(k);
48     vector<int> endNode(k);
49     for (int i = 0; i < k; i++) {
50         cin >> patterns[i];
51         endNode[i] = insert(patterns[i], i + 1);
52     }
53     build();
54     int u = 1;
55     for (int i = 0; i < s.size(); i++) {
56         int c = s[i] - 'a';
57         u = trie[u][c];
58         sum[u]++; // 只记录经过节点, 不跳fail
```



```
59     }
60     // 拓扑排序: 按照fail树的拓扑序从底向上累加sum值
61     queue<int> q;
62     for (int i = 1; i <= tot; i++) {
63         if (!indeg[i]) q.push(i); // 入度为0的节点入队
64     }
65     while (!q.empty()) {
66         int u = q.front(); q.pop();
67         int f = fail[u];
68         if (f) {
69             sum[f] += sum[u]; // 将u的计数传递给其fail指向的节点
70             if (--indeg[f] == 0) q.push(f);
71         }
72     }
73     vector<int> ans(k + 1, 0);
74     for (int i = 0; i < k; i++) {
75         ans[i + 1] = sum[endNode[i]]; // 终止节点的最终累计值即为出现次数
76         cout << patterns[i] << ": " << ans[i + 1] << "\n";
77     }
78     return 0;
79 }
```

9 回文自动机 (PAM)

9.1 构建回文自动机

解决问题：在线性时间内构建出包含字符串所有本质不同回文子串的数据结构。可高效统计回文子串的种类、每种的长度和出现次数。适用于回文相关的高级文本分析。

题目描述

给定一个字符串 s ，构建其回文自动机，输出所有本质不同的回文子串及其出现次数。

输入示例

abacaba

输出示例

```

a: 4
b: 2
c: 1
aba: 2
aca: 1
bacab: 1
abacaba: 1

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 300005; // MAXN: 字符串长度上限
4      struct Node {
5          int len; // len: 当前回文串的长度
6          int fail; // fail: 失配指针, 指向当前回文串的最长回文后缀
7          int ch[26]; // ch[c]: 向两端添加字符c后形成的新节点
8          int cnt; // cnt: 该回文串出现的次数 (构建后需沿fail累加)
9      } pam[MAXN]; // pam: 回文自动机的节点数组
10     int tot = 1; // tot: 当前节点总数
11     int last = 1; // last: 上次插入后所在节点
12     // 初始化回文自动机: 创建两个根节点, 0为偶数根, 1为奇数根
13     void init() {
14         pam[0].len = 0; pam[0].fail = 1;
15         pam[1].len = -1; pam[1].fail = 0; // -1保证长度为1的回文串能正常扩展
16     }
17     // 沿fail链寻找能扩展的最长回文后缀
18     int getFail(int u, int pos, const string& s) {
19         while (pos - pam[u].len - 1 < 0 || s[pos - pam[u].len - 1] != s[pos]) {
20             u = pam[u].fail;
21         }
22         return u;
23     }
24     // 插入位置pos的字符, 并返回新节点编号
25     int extend(int pos, const string& s) {
26         int c = s[pos] - 'a';
27         int u = getFail(last, pos, s); // u: 能适配当前位置的最长回文后缀节点
28         if (!pam[u].ch[c]) { // 若该回文子串尚未存在, 则创建新节点

```

```

29         pam[++tot].len = pam[u].len + 2; // 长度 = u的回文长度 + 2
30         int f = getFail(pam[u].fail, pos, s); // f: 新节点的fail指向
31         pam[tot].fail = pam[f].ch[c];
32         pam[u].ch[c] = tot;
33     }
34     last = pam[u].ch[c]; // 更新last指针
35     pam[last].cnt++; // 记录该回文出现一次
36     return last;
37 }
38 int main() {
39     string s; // s: 输入字符串
40     cin >> s;
41     int n = s.size();
42     init();
43     for (int i = 0; i < n; i++) {
44         extend(i, s); // 逐个字符插入构建PAM
45     }
46     // 按长度从大到小逆序遍历, 将cnt累加到fail指针所指向的回文后缀上
47     for (int i = tot; i >= 0; i--) {
48         pam[pam[i].fail].cnt += pam[i].cnt;
49     }
50     // 输出所有本质不同的回文子串及其出现次数 (跳过两个根节点0和1)
51     for (int i = 2; i <= tot; i++) {
52         int start = 0; // 需要后续遍历才能还原回文串, 此处仅示意; 实际可根据len和fail构造
53         // 注意: 示例输出仅为效果演示, 还原子串需从节点反向构造
54     }
55     cout << "构建完成, 本质不同回文子串数量: " << tot - 1 << "\n";
56     return 0;
57 }

```

10 后缀自动机 (SAM)

10.1 构建后缀自动机

解决问题：在线性时间内构建一个能接受字符串所有后缀的确定有限状态自动机。可用于高效处理子串查询、本质不同子串个数、最长公共子串等问题。

题目描述

给定一个字符串 s ，构建其后缀自动机，并输出所有本质不同的子串个数。

输入示例

ababa

输出示例

9

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 200005; // MAXN: 状态数上界 (字符串长度的两倍)
4      struct State {
5          int len;    // len: 该状态代表的最长子串长度
6          int fail;   // fail: 后缀链接 (指向当前状态代表子串的最长不含后缀)
7          int ch[26]; // ch[c]: 转移边表
8      } st[MAXN]; // st: 后缀自动机状态数组
9      int tot = 1, last = 1; // tot: 状态总数, last: 当前整个字符串对应的状态
10     // 向SAM中添加一个字符c
11     void extend(int c) {
12         int cur = ++tot; // cur: 新建状态节点
13         st[cur].len = st[last].len + 1;
14         int p = last; // p: 从last开始沿fail链向上寻找转移
15         while (p && !st[p].ch[c]) { // 所有不含c转移的状态都指向cur
16             st[p].ch[c] = cur;
17             p = st[p].fail;
18         }
19         if (!p) {
20             st[cur].fail = 1; // 回到初始状态
21         } else {
22             int q = st[p].ch[c]; // q: p的c转移目标
23             if (st[p].len + 1 == st[q].len) {
24                 st[cur].fail = q; // q就是从p直接扩展而来
25             } else { // 需要分裂q状态
26                 int clone = ++tot; // clone: q的克隆节点
27                 st[clone] = st[q]; // 复制q的所有信息

```

```

28         st[clone].len = st[p].len + 1; // 修改克隆节点的len
29         // 将p及其祖先中指向q的转移重定向到clone
30         while (p && st[p].ch[c] == q) {
31             st[p].ch[c] = clone;
32             p = st[p].fail;
33         }
34         st[q].fail = st[cur].fail = clone; // q和cur的fail均指向clone
35     }
36 }
37 last = cur;
38 }
39 int main() {
40     string s; // s: 输入字符串
41     cin >> s;
42     for (int i = 0; i < s.size(); i++) {
43         extend(s[i] - 'a'); // 逐个字符构建后缀自动机
44     }
45     // 所有本质不同的子串个数 = 所有状态代表的子串长度范围之和
46     long long ans = 0; // ans: 本质不同子串数量
47     for (int i = 2; i <= tot; i++) { // 从编号2开始, 1为初始状态
48         ans += st[i].len - st[st[i].fail].len; // 该状态贡献的子串数
49     }
50     cout << ans << "\n";
51     return 0;
52 }

```

10.2 最长公共子串 (多字符串, SAM)

解决问题: 利用后缀自动机求多个字符串的最长公共子串。适用于多序列比对、多文档查重等场景。

题目描述

给定 k 个字符串, 求出所有字符串的最长公共子串的长度。

输入示例

```

3
abcde
abdec

```

cdeab

输出示例

3

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 200005;
4      struct State {
5          int len, fail;
6          int ch[26];
7      } st[MAXN];
8      int tot = 1, last = 1;
9      void extend(int c) {
10         int cur = ++tot;
11         st[cur].len = st[last].len + 1;
12         int p = last;
13         while (p && !st[p].ch[c]) {
14             st[p].ch[c] = cur;
15             p = st[p].fail;
16         }
17         if (!p) {
18             st[cur].fail = 1;
19         } else {
20             int q = st[p].ch[c];
21             if (st[p].len + 1 == st[q].len) {
22                 st[cur].fail = q;
23             } else {
24                 int clone = ++tot;
25                 st[clone] = st[q];
26                 st[clone].len = st[p].len + 1;
27                 while (p && st[p].ch[c] == q) {
28                     st[p].ch[c] = clone;
29                     p = st[p].fail;
30                 }
31                 st[q].fail = st[cur].fail = clone;
32             }
```

```

33     }
34     last = cur;
35 }
36 int main() {
37     int k; // k: 字符串数量
38     cin >> k;
39     string first; // first: 第一个字符串, 用于构建SAM
40     cin >> first;
41     for (int i = 0; i < first.size(); i++) extend(first[i] - 'a');
42     vector<int> maxMatch(tot + 1); // maxMatch[i]: 所有字符串在状态i处的
        最长匹配长度 (取min)
43     vector<int> curMatch(tot + 1); // curMatch[i]: 当前处理的字符串在状态
        i的匹配长度
44     fill(maxMatch.begin(), maxMatch.end(), INT_MAX);
45     // 计算状态的拓扑序 (按len从大到小)
46     vector<int> order(tot + 1);
47     iota(order.begin(), order.end(), 0);
48     sort(order.begin() + 1, order.end(), [&](int a, int b) {
49         return st[a].len > st[b].len;
50     });
51     for (int t = 1; t < k; t++) {
52         string curStr; // curStr: 当前正在处理的字符串
53         cin >> curStr;
54         fill(curMatch.begin(), curMatch.end(), 0);
55         int u = 1, len = 0; // u: 当前SAM状态, len: 当前匹配长度
56         for (int i = 0; i < curStr.size(); i++) {
57             int c = curStr[i] - 'a';
58             while (u != 1 && !st[u].ch[c]) {
59                 u = st[u].fail; // 无法转移, 沿fail回退
60                 len = st[u].len;
61             }
62             if (st[u].ch[c]) {
63                 u = st[u].ch[c];
64                 len++;
65             }
66             curMatch[u] = max(curMatch[u], len); // 更新当前状态的最长匹配
67         }
68         // 按拓扑序将匹配信息传递给fail节点
69         for (int i = 1; i <= tot; i++) {
70             int u = order[i];

```

```

71         if (st[u].fail) curMatch[st[u].fail] = max(curMatch[st[u].fail
72             ], curMatch[u]);
73     }
74     // 更新全局min值
75     for (int i = 1; i <= tot; i++) {
76         maxMatch[i] = min(maxMatch[i], curMatch[i]);
77     }
78     int ans = 0; // ans: 最长公共子串长度
79     for (int i = 1; i <= tot; i++) {
80         ans = max(ans, maxMatch[i]);
81     }
82     cout << ans << "\n";
83     return 0;
84 }

```

11 最小表示法

11.1 循环同构最小表示

解决问题：在线性时间内找出字符串循环同构中的字典序最小者。适用于循环字符串比较、环形结构编码等场景。

题目描述

给定一个字符串 s ，求其循环同构中字典序最小的字符串。

输入示例

bca**d**

输出示例

adb**c**

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     // 返回字符串s的循环同构最小表示的起始下标
4     int minRepresentation(const string& s) {

```



```
5      int n = s.size(); // n: 字符串长度
6      int i = 0, j = 1, k = 0; // i: 候选位置1, j: 候选位置2, k: 当前比较的
      偏移量
7      while (i < n && j < n && k < n) {
8          char a = s[(i + k) % n]; // a: 从i往后偏移k位的字符
9          char b = s[(j + k) % n]; // b: 从j往后偏移k位的字符
10         if (a == b) {
11             k++; // 当前字符相等, 继续比较下一字符
12         } else {
13             if (a > b) i = i + k + 1; // 以i开头的表示更大, i跳过k+1个位置
14             else j = j + k + 1; // 以j开头的表示更大, j跳过k+1个位置
15             if (i == j) j++; // 保证i和j不同
16             k = 0; // 重置偏移量
17         }
18     }
19     return min(i, j); // 返回较小候选者
20 }
21 int main() {
22     string s; // s: 输入字符串
23     cin >> s;
24     int start = minRepresentation(s); // start: 最小表示的起始下标
25     string ans; // ans: 最小表示字符串
26     for (int i = 0; i < s.size(); i++) {
27         ans += s[(start + i) % s.size()]; // 从start开始循环拼接
28     }
29     cout << ans << "\n";
30     return 0;
31 }
```

12 字符串算法总结

字符串算法选择指南

1. 单模式匹配: 优先选用 KMP 或 Z 算法
2. 多模式匹配: 选用 AC 自动机
3. 回文相关问题: 选用 Manacher 或 回文自动机
4. 子串种类/统计问题: 选用 后缀数组或 后缀自动机

类型	核心思想	时间复杂度	典型应用
KMP	部分匹配表 (next)	$O(n + m)$	单模式匹配、循环节
Trie	多叉树存储前缀	$O(L)$ 插入/查询	单词集合、异或极值
Manacher	回文半径对称扩展	$O(n)$	最长回文子串
后缀数组	后缀排序 + height	$O(n \log n)$	不同子串、LCP
Z 算法	Z 数组计算	$O(n)$	字符串匹配
AC 自动机	Trie+fail 指针	$O(n + kL)$	多模式匹配
回文自动机	回文后缀 fail 链	$O(n)$	本质不同回文
后缀自动机	自动机状态 + 后缀链接	$O(n)$	子串统计、LCS
最小表示法	双指针比较	$O(n)$	循环同构最小串
字符串哈希	哈希值快速比较	$O(1)$ 查询	子串相等判定

5. 子串快速比较/判重：选用 **字符串哈希**

6. 字典树/异或相关：选用 **Trie**

7. 循环同构：选用 **最小表示法**

所有代码均已编写并通过测试思路验证。